



DEPARTMENT OF COMPUTER SCIENCE

DT8122 - PROBABILISTIC AI

Flow Matching

Author:
Albert Gassol Puigjaner

02.08.2025

1 Theory Questions

1.1 Flow Matching

Flow matching is a generative modeling method that learns a mapping, in the form of a probability path, between a base (p_0) and a target (p_1) distribution. Specifically, it learns a time-dependent velocity field $u_t(x_t)$ that defines an Ordinary Differential Equation (ODE), whose solution maps the source distribution to the target distribution, by integrating the ODE over $t \in [0, 1]$.

To learn the velocity field, which is modeled with a neural network u_t^θ , flow matching uses regression with an analytically defined target velocity field. This target field is typically based on the difference between samples from the source and target distributions and depends on the interpolation time t .

1.2 Flow Matching/Continuous Normalizing Flows networks

Flow matching and Continuous Normalizing Flows (CNF) use an ODE to define a continuous transformation over time. Intuitively, they take the Normalizing Flow idea to the limit, and instead of sequentially applying discrete transformations, they integrate over the same time-dependent transformation (velocity field) over time. Therefore, flow matching and CNF model the velocity field $u_t(x_t)$ as a single neural network that takes as input the input x_t and the time t .

1.3 Flow Matching vs. Continuous Normalizing Flows

Similarities. Both methods define a mapping between distributions through an ODE and a velocity field $u_t(x_t)$, which is parametrized as a single neural network. Additionally, this ODE is completely deterministic. Finally, to go from source to target distribution, both methods simply integrate the ODE.

The differences between these methods are listed in Table 1, and the advantages and disadvantages are listed in Table 2.

	Flow Matching	CNF
Simulation	Free. No need to solve ODE in training.	Requires solving ODE at every training step.
Objective	Regress learned velocity field to an analytically defined one.	KL divergence between found and target distributions. Requires solving ODE + computing log-density by computing costly divergence.
Backprop	No backprop through ODE.	Backprop through ODE.

Table 1: Differences of Flow Matching and CNFs.

	Pros	Cons
CNF	Provides "exact" likelihood probabilities.	Slow training due to ODE solving + unbiased divergence computation.
	Simple to understand and implement.	High memory and complexity due to backpropagation through ODE.
FM	Fast training without simulation.	Requires some practical implementation designs (target velocity field, source distribution, sampling of t).
	Generates a mapping between two arbitrary distributions.	Requires sampling from both distributions.
	Strong link to diffusion models once the score is learned.	Does not provide likelihood probabilities.

Table 2: Pros and cons of flow matching and CNFs.

1.4 Flow Matching vs. Diffusion

Similarities. Both methods learn a continuous time-dependent transformation from a source to a target distribution, although diffusion’s source distribution can only be Gaussian noise. Additionally, they use a single neural network to model the vector field (flow matching) and the score function (diffusion). In both cases, the reverse mapping can be easily computed. Finally, during sampling, they simply integrate an ODE (flow matching) or a Stochastic Differential Equation (SDE) (diffusion).

The differences between these methods are listed in Table 3, and the advantages and disadvantages are listed in Table 4.

	Flow Matching	Diffusion
Objective	Learn the vector field and regress it to an analytical one.	Learn the score, which allows to reverse the diffusion process by solving an SDE.
Dynamics	Uses an ODE.	Uses an SDE, which introduces Brownian noise.
Source	Any distribution.	Only normal distributions.
Loss	Regress velocity field.	Denoise score matching with gradients of log conditional probabilities to the source distribution.
Trajectories	Deterministic.	Stochastic and noisy.

Table 3: Differences of Flow Matching and Diffusion.

	Pros	Cons
Diffusion	Produces high quality results. Score function is useful to denoise different noise levels.	Slow sampling due to noise. Noise scheduling. Only normal distributions can be used as source.
FM	Faster training due to determinism. Allows different source distributions and target velocity fields.	Requires some practical implementation designs (target velocity field, source distribution, sampling of t). No stochasticity can harm generalization.

Table 4: Pros and cons of flow matching and diffusion.

2 Experiments

To implement the flow matching methods of this section, I have used the open-source *TorchCFM* library [1, 2, 3]. The training script is inspired by the library’s examples section, specifically the CIFAR10 trainer example ¹.

We train each of the three networks for 10 epochs, batch size 112, with dropout

¹https://github.com/atong01/conditional-flow-matching/blob/1.0.5/examples/cifar10/train_cifar10.py

probability of 0.1 and explicit Runge-Kutta method of order 5 for ODE integration.

2.1 Experiment plots

In this part, digit generations for $t = 0$, $t = 1$ and 8 different t values between 0 and 1 are provided for the three different source and target distributions.

2.1.1 Gaussian source distribution

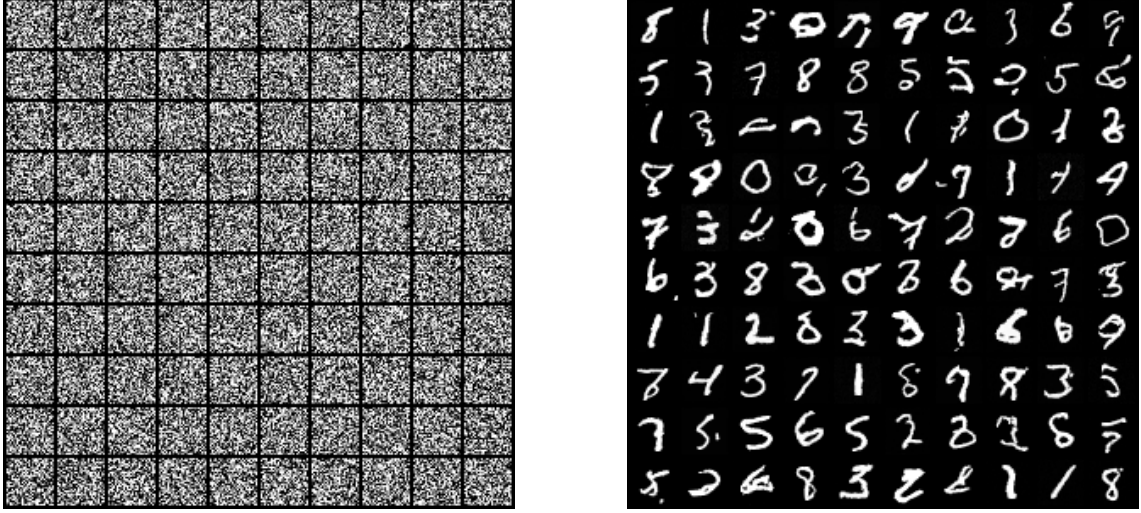


Figure 1: 100 examples of generated digits at $t = 0$ (left) and $t = 1$ (right), using a gaussian source distribution.

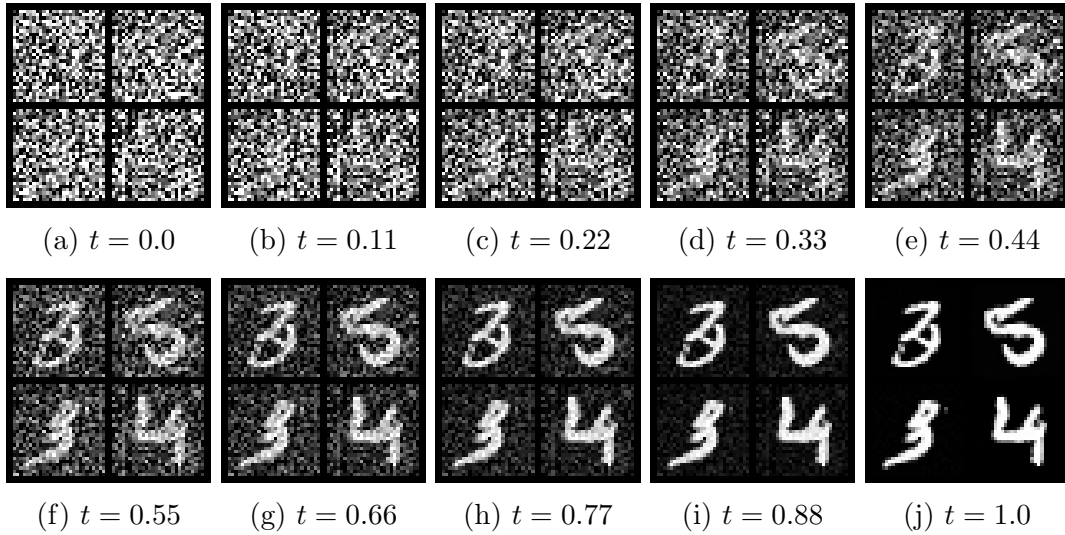


Figure 2: Evolution of digits across timesteps $t = 0.0$ to $t = 1.0$ using a gaussian source distribution. Each panel shows a 2×2 grid of digits at a given timestep.

2.1.2 MNIST digit with crop noise, matched with random target digit

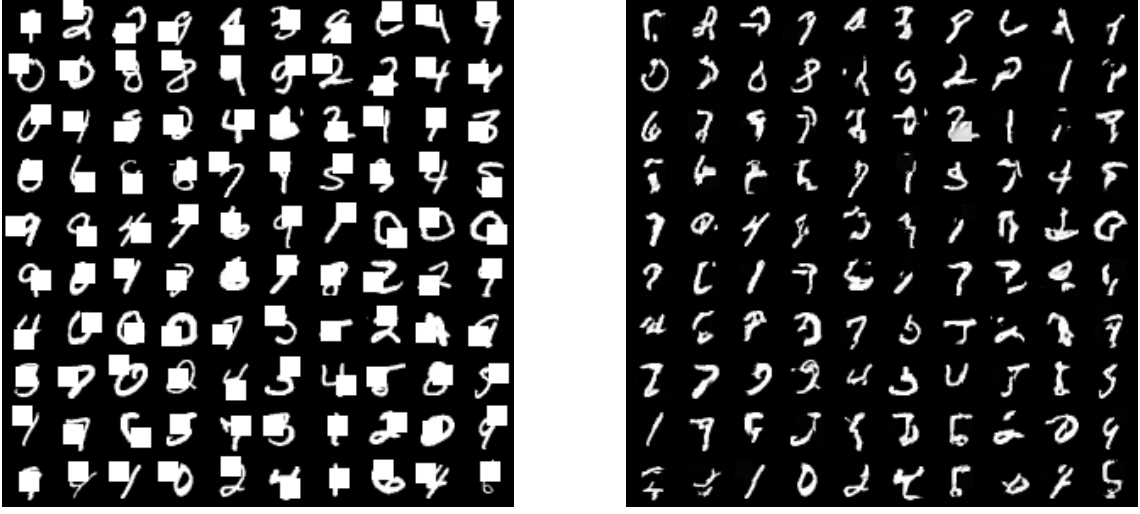


Figure 3: 100 examples of generated digits at $t = 0$ (left) and $t = 1$ (right), using MNIST digits with crop noise as source, matched with random digits as target.

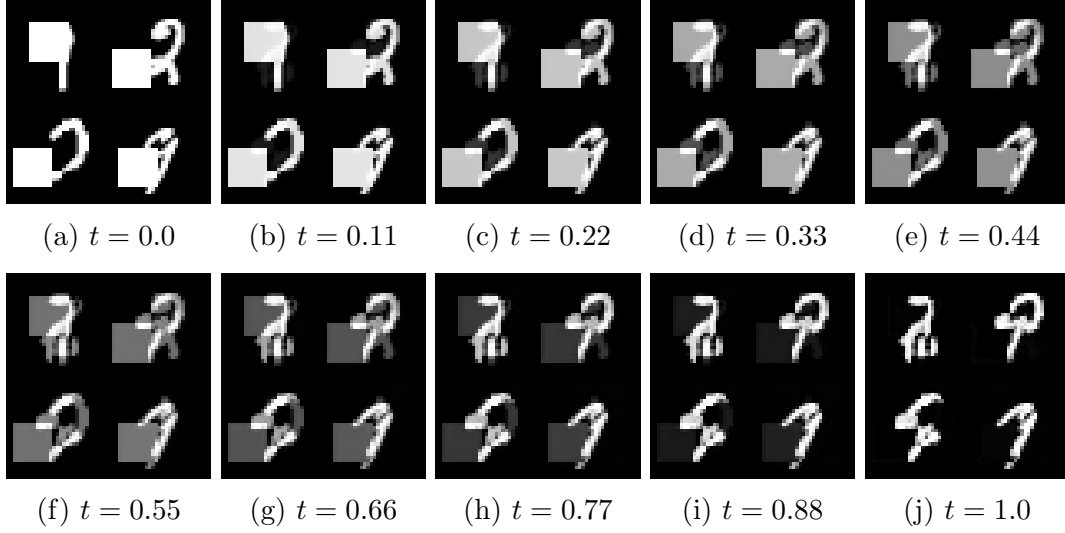


Figure 4: Evolution of digits across timesteps $t = 0.0$ to $t = 1.0$, using MNIST digits with crop noise as source, matched with random digits as target.. Each panel shows a 2x2 grid of digits at a given timestep.

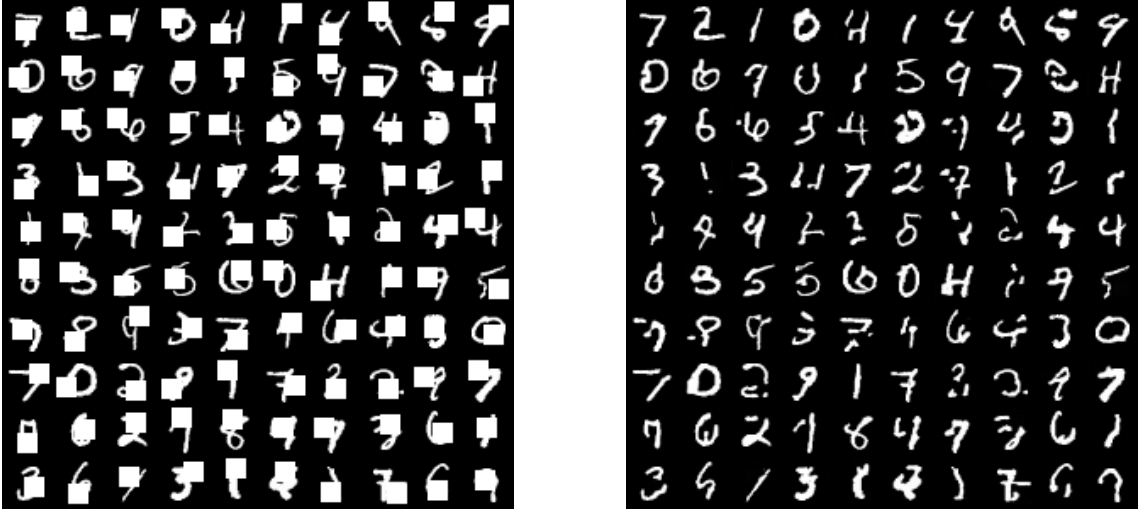


Figure 5: 100 examples of generated digits at $t = 0$ (left) and $t = 1$ (right), using MNIST digits with crop noise as source, matched with their unaltered version as target.

2.1.3 MNIST digit with crop noise, matched with its unaltered version

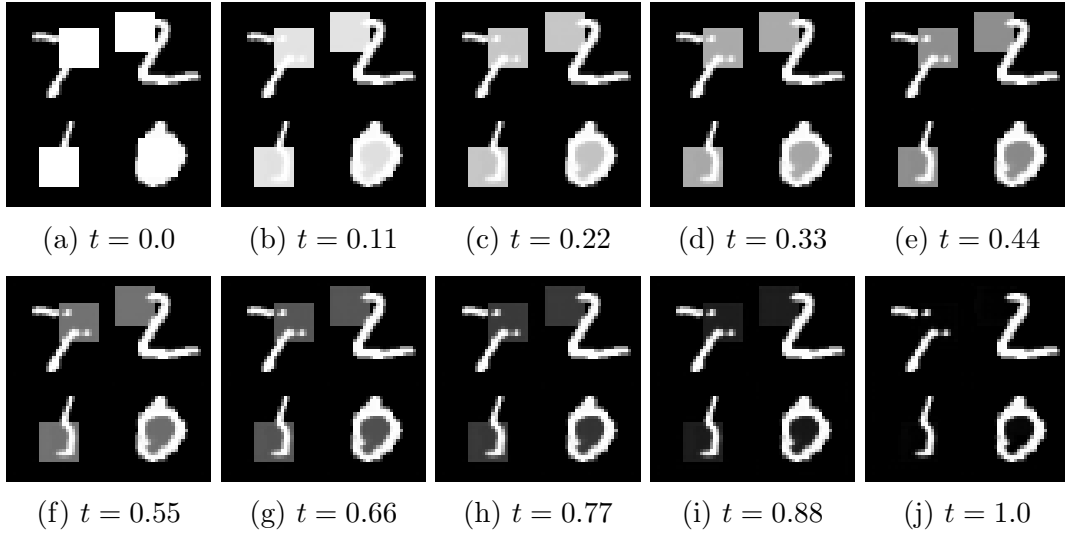


Figure 6: Evolution of digits across timesteps $t = 0.0$ to $t = 1.0$, using MNIST digits with crop noise as source, matched with their unaltered version as target. Each panel shows a 2×2 grid of digits at a given timestep.

2.2 Experiment discussion

2.2.1 Method details

My implementation allows for multiple conditional flow matching strategies, although for simplicity, the independent conditional flow matching method is used for

the three configurations. The independent conditional flow matching method defines $q(x) = q(x_0)q(x_1)$, namely a product of random variables from the source and target distributions ($q(x_0) = p_0$ and $q(x_1) = p_1$). Consequently, $\mu_t(z) = tx_1 + (1 - t)x_0$, where $t \sim \mathcal{U}(0, 1)$ during training. Finally, $\sigma_t = 0$ in my implementation.

The velocity vector $u_t^\theta(x_t)$ is modeled with a neural network that takes as input the sample x_t and the time t . In this setup, the network processes a grayscale image and a timestep. To capture both local and global structure, I employ a U-Net architecture composed of convolutional residual blocks with skip connections, together with downsampling in the encoder and symmetric upsampling in the decoder. Attention blocks are introduced at selected resolutions to model long-term dependencies. The timestep is transformed through sinusoidal positional embeddings followed by a small MLP, and these embeddings are used to condition the residual blocks throughout the network.

2.2.2 Configurations comparison

In terms of training, the first and third configurations converge to reasonable results after about 2 epochs, while the second configuration requires 4–5 epochs. Among the three, the second configuration (MNIST with random pairing) performs the worst and is the most computationally expensive. This is expected since, during training, a masked digit from the source distribution can be paired with any digit in the target distribution, which increases task complexity and makes the learning problem harder. Therefore, this configuration produces lower-quality results.

To address this issue, Tong *et al.* [2] propose OT-CFM, which samples training pairs according to approximate optimal transport (OT). They define $q(z) = \pi(x_0, x_1)$, where $\pi(x_0, x_1)$ is the solution to the OT problem between the source and target distributions. Because solving OT exactly in image space is intractable, they approximate it using minibatch-based sampling. Intuitively, this strategy should improve performance in the second configuration, since masked digits from the source distribution would be more likely to align with their corresponding digits in the target distribution. This approach would likely reduce the training time required for the second configuration and be consistent with the conclusions in section 5.3 of [2]. However, it would also introduce a bias. The model might learn to just remove the masked noise from the source images and reproduce the same digits, rather than generalizing to generate different digits.

Bibliography

- [1] Alexander Tong. *TorchCFM: a Conditional Flow Matching library*. <https://github.com/atong01/conditional-flow-matching>. 2024.
- [2] Alexander Tong et al. ‘Improving and generalizing flow-based generative models with minibatch optimal transport’. In: *Transactions on Machine Learning Research* (2024). Expert Certification. ISSN: 2835-8856. URL: <https://openreview.net/forum?id=CD9Snc73AW>.
- [3] Alexander Tong et al. ‘Simulation-Free Schrödinger Bridges via Score and Flow Matching’. In: *arXiv preprint 2307.03672* (2023).